
Mathematical Foundations of Big Data Engineering

Comprehensive Synthesis Exercise

Sessions 1–4: Distributed Systems, Kafka, Spark, Data Lakes

Student Version

Big Data Engineering – XICS404

Academic Year 2025–2026

Document Overview

This synthesis exercise integrates the four mathematical building blocks covered in Sessions 1 through 4 of the Big Data Engineering course. Through a single realistic scenario—the data platform of a global e-commerce company **GlobalCart**—you will reason quantitatively across the entire pipeline:

- **Part I.** Ingestion layer: Kafka partitioning, replication, quorums, Little’s Law (*Session 1*).
- **Part II.** Producer/Consumer reliability: acknowledgements, retries, delivery semantics, lag (*Session 2*).
- **Part III.** Processing layer: Spark parallelism, Amdahl’s Law, windowing, watermarks, cost models (*Session 3*).
- **Part IV.** Storage layer: column pruning, predicate pushdown, partition design, the small-file problem (*Session 4*).
- **Part V.** End-to-end synthesis: combining the four layers into a single capacity-and-cost decision.

Instructions. Each question is self-contained: every numerical value needed is given. Use clear formulas, comment on each result, and state your assumptions when appropriate. Express intermediate results to 3 significant digits unless otherwise specified.

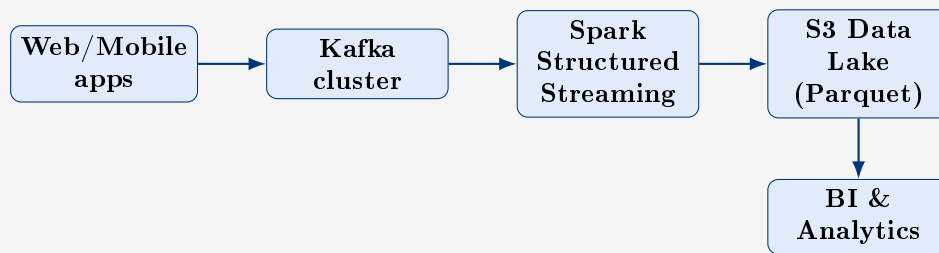
Scenario: The *GlobalCart* Data Platform

GlobalCart is a global online retailer operating in 25 countries. Its platform produces three categories of events:

- *Clickstream*: user browsing events (page views, clicks).
- *Orders*: purchase events that **must not be lost** (financial impact).

- *Telemetry*: backend service logs and metrics.

The data flows through a unified stack:



You are the lead data engineer. Your job is to size, tune, and budget each layer of this stack.

1. Part I – Ingestion Layer: Kafka Cluster Sizing

Context

The Kafka cluster has $N = 7$ brokers. Each broker has an independent probability $p = 0.01$ of being unavailable during any given hour. The `orders` topic has $P = 16$ partitions and replication factor $r = 3$ with `min.insync.replicas = 2`. Orders are keyed by `customer_id`.

1.1 Q1 – Hash partitioning and load balancing

During a one-hour window, the platform receives $n = 4\,800\,000$ orders. Assume `customer_id` values are uniformly distributed (i.e. perfect hashing).

- Compute the expected number of orders per partition $\mathbb{E}[X_i]$ and the standard deviation $\sigma(X_i)$.
- Compute the coefficient of variation $\sigma(X_i)/\mathbb{E}[X_i]$ and comment on the quality of the load balance.
- During the Black Friday sale, 25% of all orders come from a single corporate buyer (B2B account). What is the expected load of the partition holding that buyer's orders, and how does it compare to the average? What practical problem does this create?

1.2 Q2 – Replication and reliability

- Compute the probability that a single partition is *lost* during a one-hour window (i.e. all $r = 3$ replicas unavailable simultaneously).
- Compute the probability that a single partition becomes *read-only* (writes blocked because fewer than `min.insync.replicas` are up, but data is not lost).
- The `orders` topic has $P = 16$ partitions. Using the union bound, give an upper bound on the probability that *at least one partition* is lost in a given hour.

1.3 Q3 – Quorums and consistency

GlobalCart also runs a Cassandra cluster for customer-session storage with $N_c = 5$ replicas per key. The product team needs **strong consistency** on read.

- State the quorum inequality that R and W must satisfy.
- List all integer pairs (R, W) with $1 \leq R, W \leq 5$ that satisfy strong consistency.
- Among those pairs, identify the one that minimises the *maximum* of R and W (best worst-case latency). Justify why this is the standard choice in practice.
- During a network partition that isolates 2 of the 5 replicas, which configurations remain available for reads? For writes? Interpret in terms of the CAP theorem.

1.4 Q4 – Little's Law and in-flight events

- The producer rate is $\lambda = 50\,000$ events/second and the average end-to-end latency through Kafka is $L = 40$ ms. Using Little's Law, compute the average number of events $\bar{N}$ *in flight* at steady state.
- Each in-flight event consumes approximately 2 KB of memory. Estimate the total memory budget needed across the producer and broker buffers.

2. Part II – Producer & Consumer Reliability

Context

The **orders** topic is the most critical: a lost order means a lost customer. The producer is configured for maximum reliability, the consumer group writes each order to a downstream PostgreSQL database, and the team must reason quantitatively about delivery semantics.

2.1 Q5 – Producer acknowledgements

Each broker has independent failure probability $p = 0.005$ during a single write window. The replication factor is $r = 3$.

- Compute the loss probability for a single record under: `acks=1`; `acks=all` with `min.insync.replicas = 2`; `acks=all` with `min.insync.replicas = 3`.
- At a sustained rate $\lambda = 50\,000$ orders/s, compute the expected number of lost orders per *hour* under each configuration. Comment on which configurations are acceptable for a financial application.

2.2 Q6 – Retries (geometric model)

A transient send error occurs on each individual attempt with probability $q = 0.03$. Each attempt takes $L_0 = 8$ ms; the exponential back-off between attempts is $B = 100$ ms.

- Compute the expected number of attempts $\mathbb{E}[T]$ to deliver a single record.
- Compute the expected total latency $\mathbb{E}[\text{latency}]$ for delivering one record.
- The producer is configured with `retries = R = 4` (so $R + 1 = 5$ total attempts). Compute the residual loss probability q^{R+1} .
- Combine Q5(a) (`acks=all`, $m = 2$) and Q6(c): what is the overall probability that a record is ultimately lost?

2.3 Q7 – Delivery semantics

- Recall the 2×2 classification of delivery semantics. Fill in the missing cells:

	No duplicates	Possible duplicates
No loss	_____	_____
Possible loss	_____	(useless)

- The consumer is configured with the default behaviour (commit *after* processing). Under at-least-once delivery, the probability a consumer crashes between processing and commit is $\alpha = 2 \times 10^{-4}$ per record. Compute the expected number of times each record is processed.
- At $\lambda = 50\,000$ records/s, how many duplicate processings occur per hour on average?
- Since each order also triggers a debit on a non-idempotent SQL database, justify mathematically (in one or two sentences) why the team must migrate to exactly-once semantics.

2.4 Q8 – Consumer-group capacity and lag

The consumer group reading **orders** has C consumers; each consumer processes up to $\mu = 5\,000$ records/s. The topic has $P = 16$ partitions.

- (k) With $C = 8$ consumers, how many partitions does each consumer handle? What is the aggregate consumer throughput? Is the system stable at $\lambda = 50\,000$ records/s?
- (l) A Black Friday spike doubles the producer rate to $\lambda' = 100\,000$ records/s for $T_{\text{spike}} = 60$ s. Compute the lag N_0 accumulated at the end of the spike, assuming $C = 8$ consumers.
- (m) After the spike, the producer rate returns to $\lambda = 50\,000$. Compute the catch-up time $T_{\text{catch-up}}$ needed to drain the backlog N_0 .
- (n) What is the maximum useful number of consumers for this topic? Justify with the effective-parallelism formula.
- (o) If the team wants to sustain $\lambda^* = 120\,000$ records/s with the same μ , what is the minimum number of partitions P^* they should reconfigure the topic with?

3. Part III – Processing Layer: Spark Streaming

Context

A Spark Structured Streaming job reads from Kafka and computes a rolling *revenue per country* aggregate. The cluster has $E = 25$ executors with $k = 4$ cores each, and 16 GB memory each.

3.1 Q9 – Spark parallelism and Amdahl's Law

A batch reprocessing job reads $T = 600$ GB of historical data split into $P_0 = 4800$ HDFS blocks of 128 MB.

- Compute the total number of parallel slots S .
- How many waves of parallel tasks are required to process all $P_0 = 4800$ partitions?
- Each task takes $t_{\text{task}} = 8$ s. Compute the total stage time T_{stage} .
- The team plans to coalesce down to $P_1 = 200$ partitions. Assuming task time scales linearly with partition size, compute the new T_{stage} . Is the coalesce useful? Compute the per-task data size for both layouts and identify any concern.
- The job has a parallelisable fraction $f = 0.92$. Compute the speed-up at S slots using Amdahl's Law, and the theoretical maximum speed-up as $S \rightarrow \infty$.
- The team is considering doubling the cluster to $S' = 2S$ slots. Compute the new speed-up and the *relative* improvement compared to S . Comment on whether the doubling is cost-effective.

3.2 Q10 – Windowing and watermarks

The streaming job uses a **sliding window** of length $w = 15$ minutes and slide $s = 3$ minutes, with a watermark threshold $\theta = 5$ minutes. There are $K = 25$ countries (one aggregate per country per window). Each window aggregate is 32 bytes.

- How many overlapping windows does each event belong to?
- How many open windows does the engine maintain at any time?
- Estimate the total state memory required (in MB).
- Suppose the team switches to a *tumbling* 15-minute window. By what factor does the state memory shrink?
- Event lateness D is approximately exponential with mean $\mathbb{E}[D] = 60$ s. Compute the rate parameter μ , and the probability that a random event is dropped by the current watermark $\theta = 5$ min.
- The product team requires a drop rate of at most $\alpha = 10^{-4}$. Compute the smallest watermark threshold θ^* that meets this target. Express the result in minutes.

3.3 Q11 – Batch-vs-streaming cost crossover

The streaming job costs $r_s = \$0.30$ per minute to keep alive. A batch alternative would re-run the same aggregation every T minutes at a cost of $C_b = \$12$ per run.

- Compute the refresh-interval threshold T^* at which streaming becomes cheaper than batch.

- (n) The product team requires the dashboard to be no more than 10 min stale. Which option is cheaper, and by how much per hour?
- (o) How much money per month (30 days) is saved by the cheaper option, assuming 24/7 operation?

4. Part IV – Storage Layer: Data Lake

Context

Aggregated results are written to an S3 data lake in Parquet format. The raw event archive (one year of clickstream) has $V = 4$ TB uncompressed. The schema has $n = 40$ columns of average width $w = 12$ bytes per row ($W = 480$ bytes/row), with $N = 8.3 \times 10^9$ rows. Parquet uses row groups of $G = 100\,000$ rows.

4.1 Q12 – Column pruning and compression

The dataset is stored in two competing formats:

- CSV + gzip: $r_{\text{row}} = 3$;
- Parquet + Snappy: $r_{\text{col}} = 6$.

A typical query reads $|S| = 6$ columns (out of 40) to compute a country-level KPI.

- Compute the column-pruning ratio ρ_{prune} .
- Compute the bytes read by a CSV scan (compressed).
- Compute the bytes read by a Parquet scan with column pruning (no predicate pushdown).
- Compute the combined speed-up of Parquet over CSV for this query.

4.2 Q13 – Predicate pushdown

The query also has a filter `WHERE country = 'FR'`. The selectivity is $\sigma = 1/25$ (uniform across countries).

- Compute the probability that a random row group is skipped thanks to its min/max statistics (assume `country_code` values are unsorted).
- Compute the total number of row groups in the dataset.
- Now suppose the data is sorted by `country` before writing. What fraction of row groups must be read?
- Compute the bytes read by Parquet with column pruning *plus* sorted predicate pushdown. Compare to Q12(c).

4.3 Q14 – Partition design and pruning

The lake is partitioned by `country / year / month / day` with cardinalities $c_1 = 25$, $c_2 = 3$, $c_3 = 12$, $c_4 = 31$.

- Compute the total number D of leaf directories.
- A typical query reads transactions for France in 2024, between March 1 and March 7 (i.e. 7 days). Compute the partition-pruning ratio ρ_{read} .
- Combine Q13(c) and Q14(b) to compute the overall fraction of the dataset effectively scanned.

4.4 Q15 – The small-file problem

S3 has fixed per-file overhead $\tau = 100$ ms and streaming throughput $\beta = 200$ MB/s. Each day-leaf directory currently holds 2 GB of compressed data, split into 250 files of average size 8 MB.

- (l) Compute the optimal average file size s_{opt} . Are the current 8 MB files above or below this threshold?
- (m) For the query in Q14(b) (France, 7 days), how many files F must be read with the current layout?
- (n) Compute the total read time T_{read} for the query under the current layout.
- (o) The team runs a compaction job that rewrites each day-partition as a single 2 GB file. Compute the new total read time and the speed-up achieved.

4.5 Q16 – The high-cardinality trap

A product manager proposes adding `user_id` (cardinality 5×10^6) as a partition column between `country` and `year`.

- (p) Compute the new total number of leaf directories.
- (q) Assuming the dataset size remains $V = 4$ TB and each leaf holds at least one file, estimate the average file size in each leaf.
- (r) Justify mathematically why this proposal must be rejected. (Hint: compare to s_{opt} and to the per-leaf metadata cost.)

5. Part V – End-to-End Synthesis

Context

The CTO asks for a single coherent capacity plan covering the entire stack. Use the results from Parts I–IV.

5.1 Q17 – Combined throughput budget

Recall: $\lambda = 50\,000$ orders/s, $P = 16$ Kafka partitions, $\mu = 5\,000$ records/s per consumer, $C_{\max} = P$.

- What is the maximum sustainable throughput $\lambda_{\max}$ of the current stack? Where is the bottleneck?
- The marketing team forecasts +60% growth next year. Propose a configuration (new P^* , new C^*) that sustains the forecast. Estimate the cost impact at $r_s = \$0.30/\text{min}$ and a per-consumer rate of $\$0.05/\text{minute}$.

5.2 Q18 – End-to-end reliability budget

The platform has three reliability stages: Kafka producer (Q6(d)), Spark processing (assume $\varepsilon_{\text{Spark}} = 10^{-6}$ per record), and S3 writes (assume $\varepsilon_{\text{S3}} = 10^{-7}$ per record).

- Assuming the three stages fail independently, compute the end-to-end probability that a single order is lost.
- At $\lambda = 50\,000$ orders/s, what is the expected number of lost orders per year? Comment on whether this meets a 5-nines reliability target (99.999%).

5.3 Q19 – End-to-end latency budget

Each stage contributes to the total user-perceived latency: Kafka in-flight = 40 ms (Q4(a)), Spark micro-batch interval $\Delta t = 5$ s, S3 commit latency = 200 ms.

- Compute the expected end-to-end latency from user click to BI dashboard.
- The product manager demands sub-second latency. Which stage dominates, and what configuration change would you propose first?

5.4 Q20 – End-to-end cost-and-storage

The dataset grows by $V_{\text{daily}} = 12$ GB per day uncompressed. Storage costs: S3 at $p = \$0.023/\text{GB-month}$.

- Compute the monthly storage cost in CSV+gzip ($r_{\text{row}} = 3$) vs. Parquet+Snappy ($r_{\text{col}} = 6$) vs. Parquet+Zstd ($r = 10$). Assume a year of accumulated data.
- Across the four layers (Kafka, Spark, S3 storage, S3 access), where should investment focus to reduce total cost? Defend your answer in 3–5 sentences using results from previous questions.